

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 2 การออกแบบขั้นตอนวิธี รหัสจำลอง และผังงานมาตรฐานสากล

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา

สังกัด สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

เอกสารประกอบรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ

แผนบริหารการสอนประจำบทที่ 2

1. หัวข้อเนื้อหาประจำบท

1. เรื่องเล่าเปิดบทเรียนและภาษาภาพแห่งความคิด โศกนาฏกรรมยานอวกาศ Mars Climate Orbiter (1999) กับความสำคัญของการสื่อสารขั้นตอนวิธีที่ไม่กำกวม
2. **รากฐานการออกแบบขั้นตอนวิธี** นิยาม คุณสมบัติ 5 ประการของอัลกอริทึม และระดับของภาษาการสื่อสาร
3. **มาตรฐานการเขียนรหัสจำลองสากล** โครงสร้างไวยากรณ์คำสำคัญ `INPUT` , `OUTPUT` , `COMPUTE` , `IF-THEN-ELSE` , `WHILE` , `FOR`
4. สัญลักษณ์ผังงานมาตรฐาน ISO 5807 / ANSI: ความหมาย การใช้งาน และข้อกำหนดทางวิศวกรรมซอฟต์แวร์
5. 3 โครงสร้างการควบคุมหลัก (Three Fundamental Control Structures):
 - โครงสร้างแบบเรียงลำดับ (Sequential Structure)
 - โครงสร้างแบบทางเลือกและการตัดสินใจ (Selection / Branching Structure)
 - โครงสร้างแบบวนซ้ำ (Iteration / Repetition Structure)
6. **การตรวจสอบความถูกต้องด้วยตารางตรรกะ** เทคนิคการติดตามค่าตัวแปรในหน่วยความจำที่ละบรรทัด และอัลกอริทึมของยูคลิด (Euclidean Algorithm)
7. การประยุกต์ใช้โปรแกรมคอมพิวเตอร์ การเขียนโปรแกรมภาษา Python 3.11 จำลองกลไกผังงานและสร้าง Trace Table อัตโนมัติ
8. คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe: การต่อบล็อกผังงานในอวกาศ 3 มิติและการจำลองลำแสงเลเซอร์ตัดสินใจ

2. วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. ****อธิบาย **** นิยาม ความสำคัญ และบทบาทของขั้นตอนวิธี รหัสจำลอง และผังงานมาตรฐานในการพัฒนาซอฟต์แวร์ทางวิทยาศาสตร์ได้อย่างถูกต้อง
2. ****เขียน **** รหัสจำลองตามมาตรฐานสากลเพื่อแก้ปัญหาทางคณิตศาสตร์และวิทยาศาสตร์ได้อย่างเป็นระบบ
3. ****ออกแบบและวาด **** ผังงานตามมาตรฐาน ISO 5807 โดยใช้โครงสร้างแบบเรียงลำดับ ทางเลือก และวนซ้ำได้อย่างแม่นยำ
4. ****วิเคราะห์และตรวจสอบ **** ข้อผิดพลาดของขั้นตอนวิธีด้วยตารางตรารัน (Trace Table) ที่ละขั้นตอนได้อย่างถูกต้อง 100%
5. ****สร้างสรรค์ **** โปรแกรมภาษา Python 3.11 ที่แปลงผังงานและการวนซ้ำไปสู่การทำงานจริงบนคอมพิวเตอร์ได้
6. ****ปฏิบัติการ **** การทดลองเสมือนจริง 3D AR MediaPipe Hands เพื่อควบคุมทิศทางการไหลของข้อมูลในผังงานแบบไร้สัมผัสได้อย่างคล่องแคล่ว

3. กิจกรรมการเรียนรู้

- ****ขั้นนำเข้าสู่บทเรียน **** ฉายภาพจำลองวิถียาน Mars Climate Orbiter และชวนผู้เรียนวิเคราะห์ว่าทำไมข้อผิดพลาดในการแปลงหน่วยเพียงจุดเดียวจึงทำให้ยานมูลค่าหลายร้อยล้านดอลลาร์สูญสลายในชั้นบรรยากาศ
- ****ขั้นจัดกิจกรรมการเรียนรู้****
 - บรรยายสัญลักษณ์ผังงาน ISO 5807 และฝึกเขียน Pseudocode ตามโจทย์สถานการณ์
 - กิจกรรมกลุ่มย่อย (Peer Review): สลับกันตรวจสอบตาราง Trace Table เพื่อค้นหาจุดบกพร่อง (Logic Bug)
 - สาธิตการเขียนโค้ด Python ตรวจสอบขั้นตอนวิธีหา ห.ร.ม. ของยูคลิด
- ****ขั้นประยุกต์และสรุปผล ****
 - ผู้เรียนเข้าสู่ห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe ถึง ([chapter02_visual_flowchart_intro.html](#) [chapter02_trace_table_runner.html](#))
 - ทำแบบประเมินตนเอง Quick Concept Check และตอบคำถามท้ายบทเรียน

2.0 เรื่องเล่าเปิดบทเรียน บทเรียนราคาแพงจากอวกาศสู่วิศวกรรมขั้นตอนวิธี

ในวันที่ 23 กันยายน ค.ศ. 1999 องค์การบริหารการบินและอวกาศแห่งชาติสหรัฐอเมริกา (NASA) ต้องเผชิญกับหนึ่งในความสูญเสียครั้งใหญ่ที่สุดในประวัติศาสตร์การสำรวจอวกาศ เมื่อยานสำรวจดาวอังคาร Mars Climate

Orbiter (MCO) มูลค่ากว่า 327.6 ล้านดอลลาร์สหรัฐ ได้ขาดการติดต่อและพุ่งเข้าสู่ชั้นบรรยากาศของดาวอังคารในระดับความสูงที่ต่ำเกินไปจนยานเกิดการเผาไหม้และระเบิดแตกกระจาย

ข้อผิดพลาดทางตรรกะระดับประวัติศาสตร์ (Root Cause Analysis)

NASA Investigation Report

จากการสอบสวนของคณะกรรมการอิสระ พบว่าสาเหตุไม่ได้เกิดจากความล้มเหลวของเครื่องยนต์ แต่เกิดจาก ความบกพร่องในการออกแบบและการสื่อสารขั้นตอนวิธีระหว่างทีมวิศวกร:

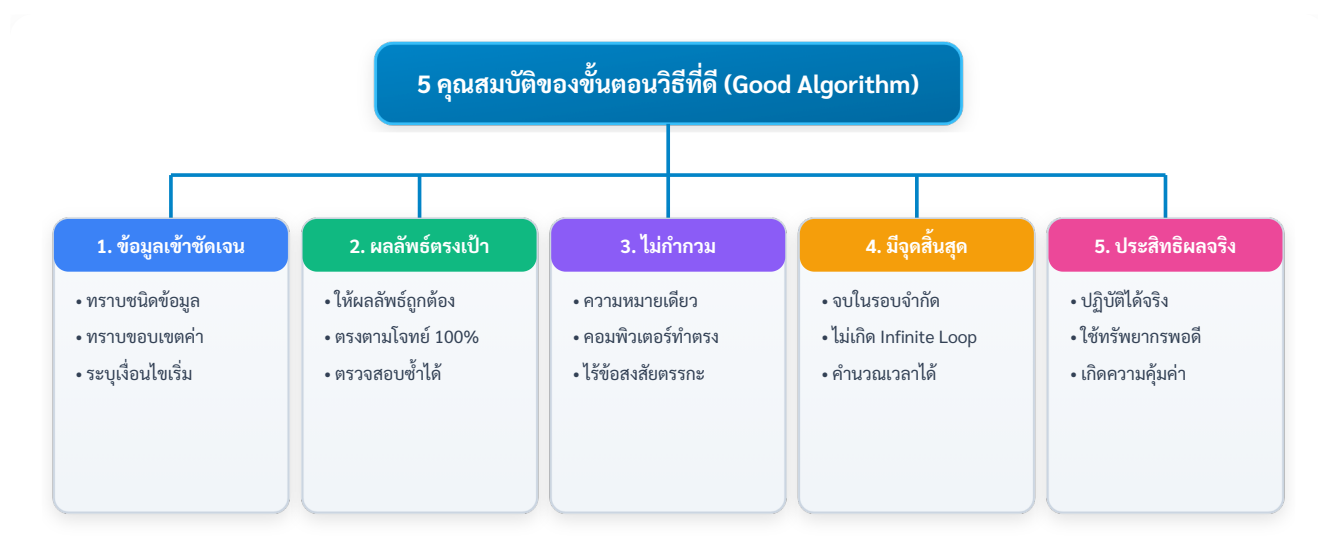
- ทีมผู้สร้างซอฟต์แวร์ภาคพื้นดิน (Lockheed Martin) ส่งค่าแรงขับเคลื่อน (Impulse) ในหน่วย ปอนด์-แรง-วินาที (Pound-force-seconds: lbf-s) ตามระบบอังกฤษ
- แต่ทีมควบคุมการบินของ NASA ออกแบบอัลกอริทึมประมวลผลโดยคาดหวังหน่วย นิวตัน-วินาที (Newton-seconds: N-s) ตามระบบเมตริกสากล (SI Metric)
- อัตราส่วนการแปลงที่ผิดพลาด $1 \text{ lbf-s} \approx 4.45 \text{ N-s}$ ทำให้นานปรับวิถีผิดพลาดสะสมจนระดับความสูงลดต่ำลงเหลือเพียง 57 km (จากเกณฑ์ปลอดภัย 150 km)

เหตุการณ์นี้กลายเป็นกรณีศึกษาที่สำคัญที่สุดในวงการวิทยาการคอมพิวเตอร์ ที่ตอกย้ำว่า

"ขั้นตอนวิธี ไม่ใช่แค่โค้ดที่โปรแกรมเมอร์เขียนขึ้น แต่คือ 'ภาษาและสัญญาทางวิศวกรรม' ที่ต้องมีความชัดเจน แม่นยำ ไม่กำกวม และมีมาตรฐานกำกับในทุกขั้นตอน"

2.1 นิยามและคุณสมบัติ 5 ประการของขั้นตอนวิธีที่ดี

ขั้นตอนวิธี คือ ชุดของคำสั่งหรือลำดับขั้นตอนการทำงานที่ถูกจัดเรียงไว้อย่างเป็นระเบียบ เพื่อใช้ในการประมวลผลข้อมูลนำเข้า (Inputs) ให้กลายเป็นผลลัพธ์ที่ต้องการ (Outputs) โดยอัลกอริทึมที่ถูกต้องตามหลักวิศวกรรมซอฟต์แวร์ต้องมีคุณสมบัติครบ 5 ประการ



ภาพที่ 2.1 แผนผังโครงสร้าง 5 คุณสมบัติของขั้นตอนวิธีที่ดีตามมาตรฐานวิศวกรรมซอฟต์แวร์

2.2 มาตรฐานการเขียนรหัสลำลองสากล

รหัสลำลอง คือ ภาษาจำลองที่มนุษย์ใช้ถ่ายทอดขั้นตอนวิธี โดยผสมผสานระหว่างโครงสร้างของภาษาคอมพิวเตอร์และภาษาธรรมชาติ (ภาษาอังกฤษ) เพื่อให้ง่ายต่อการอ่านและแปลไปสู่ภาษาโปรแกรมจริง (เช่น Python, C++, Java)

คำสำคัญมาตรฐานสากล

หมวดหมู่การทำงาน	คำสำคัญมาตรฐานสากล (Keywords)	ตัวอย่างการใช้งานในรหัสจำลอง
การเริ่มต้น / สิ้นสุด	<code>START</code> / <code>END</code> หรือ <code>BEGIN</code> / <code>END</code>	<code>START ... END</code>
การรับข้อมูลนำเข้า	<code>INPUT</code> , <code>READ</code> , <code>GET</code>	<code>INPUT sensor_temperature, humidity</code>
การแสดงผลลัพธ์	<code>OUTPUT</code> , <code>PRINT</code> , <code>DISPLAY</code>	<code>OUTPUT "Status: System Ready", current_time</code>
การคำนวณและกำหนดค่า	<code>SET</code> , <code>COMPUTE</code> , <code>CALCULATE</code> , <code>←</code>	<code>SET kinetic_energy = 0.5 * mass * velocity^2</code>
โครงสร้างทางเลือก	<code>IF ... THEN ... ELSE ... ENDIF</code>	<code>IF score ≥ 70 THEN OUTPUT "PASS" ELSE OUTPUT "FAIL" ENDIF</code>
โครงสร้างวนรอบนับ	<code>FOR ... TO ... STEP ... ENDFOR</code>	<code>FOR i = 1 TO 10 STEP 1 DO ... ENDFOR</code>
โครงสร้างวนรอบเงื่อนไข	<code>WHILE ... DO ... ENDWHILE</code>	<code>WHILE battery_level > 20 DO ... ENDWHILE</code>

2.3 สัญลักษณ์ผังงานมาตรฐาน ISO 5807 / ANSI

ผังงาน คือ แผนภาพทางเรขาคณิตที่ใช้แสดงลำดับขั้นตอน ทิศทางการไหลของข้อมูล (Data Flow) และตรรกะการตัดสินใจในระบบ โดยใช้สัญลักษณ์มาตรฐานสากล



ภาพที่ 2.2 สัญลักษณ์ผังงานมาตรฐานสากล ISO 5807 และ ANSI Standard

ตารางสรุปสัญลักษณ์ผังงาน ISO 5807

สัญลักษณ์	ชื่อภาษาไทย	ชื่อภาษาอังกฤษ	หน้าที่และการใช้งาน
	จุดเริ่มต้นและสิ้นสุด	Terminator	ระบุจุดเริ่มต้น (START) หรือจุดสิ้นสุด (END) ของโปรแกรม
	การประมวลผล	Process	การคำนวณทางคณิตศาสตร์ หรือการกำหนดค่าตัวแปร (SET x = 10)
	การรับเข้า/ส่งออกข้อมูล	Input / Output	การรับค่าจากคีย์บอร์ด/เซนเซอร์ หรือการแสดงผลออกทางหน้าจอ
	การตัดสินใจ	Decision	การตรวจสอบเงื่อนไขตรรกะ จะมีเส้นทางออกอย่างน้อย 2 ทาง (True / False)
	จุดเชื่อมต่อ	On-page Connector	เชื่อมต่อเส้นการไหลของผังงานที่มาจากหลายทิศทางให้อยู่ในจุดเดียวกัน
<i>longrightarrow</i>	เส้นแสดงทิศทาง	Flowline	ลูกศรระบุทิศทางการไหลของการประมวลผล (บนลงล่าง หรือ ซ้ายไปขวา)

2.4 โครงสร้างควบคุมการทำงาน 3 รูปแบบหลัก

ตามทฤษฎีบทของ โบห์มและจาโคปีนี (Böhm-Jacopini Theorem, 1966) ทุกขั้นตอนวิธีในโลกสามารถสร้างขึ้นได้ด้วยการผสมผสานของโครงสร้างควบคุมพื้นฐาน 3 รูปแบบเท่านั้น

1. โครงสร้างแบบเรียงลำดับ

การประมวลผลคำสั่งที่ละบรรทัดจากบนลงล่าง โดยไม่มีการข้ามขั้นตอนหรือวนซ้ำ

```
graph TD
    A["START"] → B["INPUT base, height"]
    B → C["COMPUTE area = 0.5 * base * height"]
    C → D["OUTPUT area"]
    D → E["END"]
```

2. โครงสร้างแบบทางเลือกและการตัดสินใจ

การแตกแขนงเส้นทางการทำงานตามผลลัพธ์ของเงื่อนไขตรรกะ (True หรือ False):

```
graph TD
    A["INPUT temperature"] → B{"temperature ≥ 100 ?"}
    B → |True (จริง)| C["OUTPUT 'สสารมีสถานะเป็นก๊าซ (Steam)'" ]
    B → |False (เท็จ)| D{"temperature ≤ 0 ?"}
    D → |True (จริง)| E["OUTPUT 'สสารมีสถานะเป็นของแข็ง (Ice)'" ]
    D → |False (เท็จ)| F["OUTPUT 'สสารมีสถานะเป็นของเหลว (Water)'" ]
    C → G["END"]
    E → G
    F → G
```

3. โครงสร้างแบบวนซ้ำ

การทำงานซ้ำกลุ่มคำสั่งเดิมจนกว่าเงื่อนไขที่กำหนดจะเปลี่ยนเป็นเท็จ (While Loop) หรือครบจำนวนรอบที่กำหนด (For Loop):

```
graph TD
    A["SET count = 1, sum = 0"] → B{"count ≤ 5 ?"}
    B → |True (วนรอบต่อ)| C["COMPUTE sum = sum + count"]
    C → D["COMPUTE count = count + 1"]
    D → B
    B → |False (จบloop)| E["OUTPUT 'ผลรวม = ', sum"]
    E → F["END"]
```

2.5 การตรวจสอบข้อผิดพลาดด้วยตารางรายรับ

****ตารางรายรับ**** คือ เทคนิคการจำลองการทำงานของคอมพิวเตอร์ด้วยการเขียนบนกระดาษ (Manual Execution) โดยติดตามการเปลี่ยนแปลงค่าของตัวแปรและผลลัพธ์การเปรียบเทียบเงื่อนไขที่ละบรรทัด เพื่อค้นหา ****บั๊กทางตรรกะ**** ก่อนลงมือเขียนโค้ดจริง

กรณีศึกษาตัวอย่าง ขั้นตอนวิธีหา ห.ร.ม. ของยูคลิด

โจทย์ จงหาตัวหารร่วมมาก (ห.ร.ม.) ของตัวเลข $A = 48$ และ $B = 18$ ด้วยขั้นตอนวิธีของยูคลิด

```
START
INPUT A, B
WHILE B  $\neq$  0 DO
    SET remainder = A mod B
    SET A = B
    SET B = remainder
ENDWHILE
OUTPUT "GCD = ", A
END
```

ตาราง Trace Table ติดตามการทำงานทีละรอบ

| บรรทัดที่ (Line) | ตัวแปร A | ตัวแปร B | เงื่อนไข (\$B

eq 0\$) | ตัวแปร **remainder** ($A \bmod B$) | ผลลัพธ์ที่แสดงผล (Output) | | :---: | :---: | :---: | :---: | :---: | :---: |
 -: | | เริ่มต้น | 48 | 18 | — | — | — | | รอบที่ 1 | 48 | 18 | True ($18 \neq 0$) | $48 \bmod 18 = \mathbf{12}$ | — | — |
 (สลับค่า) | $\mathbf{18}$ | $\mathbf{12}$ | — | — | — | | รอบที่ 2 | 18 | 12 | True ($12 \neq 0$) | $18 \bmod 12 =$
 $\mathbf{6}$ | — | — | (สลับค่า) | $\mathbf{12}$ | $\mathbf{6}$ | — | — | — | | รอบที่ 3 | 12 | 6 | True ($6 \neq 0$) |
 $12 \bmod 6 = \mathbf{0}$ | — | — | (สลับค่า) | $\mathbf{6}$ | $\mathbf{0}$ | — | — | — | | รอบที่ 4 | 6 | 0 | False ($0 \neq 0$ จบรูป) | — | — | — | | สิ้นสุด | $\mathbf{6}$ | $\mathbf{0}$ | — | — | — | | OUTPUT: GCD = 6 |

บทวิเคราะห์ ตาราง Trace Table พิสูจน์ว่า ห.ร.ม. ของ 48 และ 18 คือ 6 ในการวนรอบเพียง 3 รอบอย่างแม่นยำ 100%!

2.6 โค้ดคอมไพเตอร์ภาษา Python 3.11 แบบสมบูรณ์

โปรแกรมภาษา Python ต่อไปนี้ทำหน้าที่จำลองการทำงานของผังงานยูคลิด และสร้างตาราง Trace Table บันทึกค่าตัวแปรในหน่วยความจำออกมาทางหน้าจอโดยอัตโนมัติ

```
# =====
# euclidean_trace_table_simulator.py
# โปรแกรมจำลองการทำงานของผังงานและสร้างตาราง Trace Table อัตโนมัติ
# ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร.ชัชวาล ทัศนาศ (มหาวิทยาลัยราชภัฏรำไพพรรณี)
# มาตรฐาน: Python 3.11+ • PEP 8 Compliant • Pure Standard Library
# =====

from typing import List, Tuple

def euclidean_algorithm_with_trace(a_init: int, b_init: int) → Tuple[int, List[dict]]:
    """
    คำนวณหาตัวหารร่วมมาก (ห.ร.ม.) ด้วย Euclidean Algorithm พร้อมบันทึก Trace Table

    Parameters:
        a_init (int): จำนวนเต็มบวกตัวแรก
        b_init (int): จำนวนเต็มบวกตัวที่สอง

    Returns:
        Tuple[int, List[dict]]: (ค่า ห.ร.ม., ประวัติการเปลี่ยนค่าตัวแปรในตาราง Trace Table)
    """
    a = a_init
    b = b_init
    trace_history = []
    step = 1

    while b != 0:
        remainder = a % b
        condition_eval = (b != 0)

        # บันทึกสถานะก่อนสลับค่า
        trace_history.append({
            "step": step,
            "a_before": a,
            "b_before": b,
            "condition": condition_eval,
            "remainder": remainder,
            "a_after": b,
            "b_after": remainder
        })

        # ปรับปรุงค่าตามขั้นตอนวิธี
        a = b
        b = remainder
        step += 1

    return a, trace_history
```

```
def display_formatted_trace_table(a_init: int, b_init: int, gcd: int, history: L
    """แสดงผลตาราง Trace Table รูปแบบ ASCII แบบมีอานชีพ"""
    print("
" + "=" * 80)
    print(f"    ตาราง TRACE TABLE การทำงานของ EUCLIDEAN ALGORITHM: GCD({a_init},
    print("=" * 80)
    header = f"{'รอบ (Step)':<10} | {'ตัวแปร A':<10} | {'ตัวแปร B':<10} | {'เงื่อนไข
    print(header)
    print("-" * 80)

    for row in history:
        cond_str = "True (วนรอบต่อ)" if row["condition"] else "False"
        print(f"{'row['step']':<10} | {'row['a_before']':<10} | {'row['b_before']':<10}

    print("-" * 80)
    print(f"    ผลลัพธ์สุดท้าย (OUTPUT): ห.ร.ม. ของ {a_init} และ {b_init} มีค่าเท่ากับ {g
    print("=" * 80 + "
")

if __name__ == "__main__":
    # 1. รันการทดสอบและพิมพ์ตาราง Trace Table
    test_a, test_b = 48, 18
    gcd_result, trace_log = euclidean_algorithm_with_trace(test_a, test_b)
    display_formatted_trace_table(test_a, test_b, gcd_result, trace_log)

    # 2. ทำการตรวจสอบ Unit Assertions
    assert gcd_result == 6, f"Expected GCD 6, but got {gcd_result}"
    assert len(trace_log) == 3, f"Expected 3 steps, but got {len(trace_log)}"
    print("    ระบบผ่านการตรวจสอบความถูกต้องของ Assertion Tests 100% OK!
")
```

2.7 คู่มือใบงานห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands (unk 2)

เพื่อส่งเสริมการจัดการเรียนรู้เชิงรุก (Active Learning) ผู้เรียนสามารถเข้าสู่ชุดปฏิบัติการจำลองเสมือนจริงทั้งในรูปแบบ 2D Interactive Canvas และ 3D AR MediaPipe Spatial View ผ่านเว็บเบราว์เซอร์บน Global CDN โดยไม่ต้องติดตั้งโปรแกรมใดๆ เพิ่มเติม โดยมีคู่มือปฏิบัติการฉบับสมบูรณ์ที่ [lab_manual_chapter02_ar_mediapipe.md](#):

```
graph TD
    LAB["ชุด 5 ห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำบทที่ 2"]
    LAB --> L1["2.0 Mars Unit Converter & Flowchart Intro"]
    L1 --> L1_L["chapter02_visual_flowchart_intro.html"]
    L1 --> L1_L2["จำลองวิถียาน Mars ป้องกันบั๊กหน่วยวัดด้วยผังงาน ISO 5807"]
    LAB --> L2["2.1 Pseudocode AST Linter"]
    L2 --> L2_L["chapter02_pseudocode_linter.html"]
    L2 --> L2_L2["ตัวตรวจจับข้อผิดพลาดไวยากรณ์และต้นไม้ 3D AST"]
    LAB --> L3["2.2 Flowchart Builder Studio"]
    L3 --> L3_L["chapter02_flowchart_builder.html"]
    L3 --> L3_L2["สตูดิโอประกอบบล็อกผังงานและปล่อยเลเซอร์ทดสอบ"]
    LAB --> L4["2.3 Multi-Branch Decision Gates"]
    L4 --> L4_L["chapter02_decision_branching.html"]
    L4 --> L4_L2["ระบบตัดสินใจหลายเงื่อนไขควบคุมโรงเรือนเกษตรอัจฉริยะ"]
    LAB --> L5["2.4 Animated Trace Table Runner"]
    L5 --> L5_L["chapter02_trace_table_runner.html"]
    L5 --> L5_L2["ตารางไล่ค่าตัวแปรหาผลรวมอนุกรม พร้อมลูกบาศก์หน่วยความจำ 3D"]
```

ตารางสรุป 5 ปฏิบัติการเสมือนจริงและสาระสำคัญประจำบทที่ 2

รหัส แล็บ	ชื่อการทดลอง (Experiment Title)	วัตถุประสงค์เชิง พฤติกรรม	รูปแบบ การ จำลอง	ตัวแปรและ ข้อมูลที่บันทึก	ผลสรุปทางวิทยาการ คำนวณ
LAB 2.0	แบบจำลองการ แปลงหน่วยวิถียาน Mars	ศึกษาความสำคัญ ของความถูกต้องเชิง ขั้นตอนวิธี	2D/3D AR	แรงขับ (lbf/N), ความสูง h , ผล การเข้าวงโคจร	การแปลงหน่วยช่วยให้ยาน เข้าวงโคจรที่ 193extkm ปลอดภัย
LAB 2.1	ตัววิเคราะห์รหัส จำลองและต้นไม้ AST	ตรวจสอบไวยากรณ์ และบล็อกคำสั่ง	2D/3D AR	คำสั่งที่ป้อน, Error Code, สถานะ 3D Tree	การปิดบล็อก (<code>ENDIF</code>) ชัดเจนป้องกันความกำกวม ของโปรแกรม
LAB 2.2	สตูดิโอสร้างผังงาน มาตรฐาน ISO	ประกอบผังงานและ ตรวจสอบ Flowline	2D/3D AR	ลำดับบล็อก, เส้น ทางเลเซอร์, ผลลัพธ์ Output	ผังงานช่วยให้ตรวจสอบ เงื่อนไขผ่าน/ตกเกณฑ์ได้ ครบถ้วน
LAB 2.3	การตัดสินใจหลาย เงื่อนไข Smart Farm	จำลองตรรกะ ควบคุมสภาพ แวดล้อมอัตโนมัติ	2D/3D AR	อุณหภูมิ, ความชื้น, ระดับ น้ำ, ปัมพ์หมอก	เงื่อนไขความปลอดภัย ป้องกันปั้มน้ำเสียหายเมื่อ น้ำหมดถัง
LAB 2.4	การไล่ค่าตัวแปร ด้วยตารางอนิเมชัน	ตรวจสอบความถูก ต้องของลูปด้วย Dry Run	2D/3D AR	ตัวแปร i , Sum , เงื่อนไข $ile5$, ลูปสแต็บ	ได้ผลรวม $Sum = 15$ ตรงตามสูตรอนุกรม คณิตศาสตร์ $\frac{n(n+1)}{2}$

2.8 สรุปสาระสำคัญประจำบท

- ขั้นตอนวิธี เป็นหัวใจของการแก้ปัญหาเชิงคำนวณที่ต้องมีคุณสมบัติ 5 ประการ ได้แก่ ข้อมูลเข้าชัดเจน, ผลลัพธ์ตรงเป้าหมาย, ไม่กำกวม, มีจุดสิ้นสุด, และปฏิบัติได้จริง
- รหัสจำลอง เป็นเครื่องมือสื่อสารระหว่างมนุษย์ด้วยคำสำคัญสากล (`INPUT` , `OUTPUT` , `IF-THEN` , `WHILE` , `FOR`) ขณะที่ ผังงาน ใช้ภาษาภาพตามมาตรฐาน ISO 5807 เพื่อแสดงทิศทางการไหลของข้อมูล
- โครงสร้างควบคุม 3 รูปแบบ (เรียงลำดับ, ทางเลือก, วงซ้ำ) เพียงพอที่จะใช้สร้างอัลกอริทึมทุกชนิดในโลกตามทฤษฎีบท Böhm-Jacopini
- **ตารางทรายรัน** เป็นเครื่องมือสำคัญที่สุดในการตรวจสอบความถูกต้องและกำจัดบั๊ก (Debugging) ก่อนลงมือเขียนโค้ดจริง

2.9 แบบฝึกหัดทบทวนและโจทย์ประเมินผลสัมฤทธิ์

ตอนที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงอธิบายความหมายและความแตกต่างของสัญลักษณ์ผังงาน **Process** (สี่เหลี่ยมผืนผ้า), **Input/Output** (สี่เหลี่ยมด้านขนาน), และ **Decision** (สี่เหลี่ยมข้าวหลามตัด)
- จงระบุคุณสมบัติ 5 ประการของขั้นตอนวิธีที่ดี พร้อมอธิบายว่าเหตุใดคุณสมบัติ "Finiteness (ความมีจุดสิ้นสุด)" จึงสำคัญอย่างยิ่งในระบบยานอวกาศ
- จงเขียนรหัสจำลอง สำหรับรับค่ารัศมีของวงกลม (r) และคำนวณหาพื้นที่วงกลม ($Area = \pi r^2$)

ตอนที่ 2 การวิเคราะห์และประยุกต์ใช้

- **ระบบตรวจวัดดัชนีมวลกาย ****
 - กำหนดสูตรคำนวณ: $BMI = \frac{Weight (kg)}{(Height (m))^2}$
 - เกณฑ์จำแนก: $BMI < 18.5$ (น้ำหนักน้อย), $18.5 \leq BMI < 23.0$ (สมส่วน), $23.0 \leq BMI < 25.0$ (น้ำหนักเกิน), $BMI \geq 25.0$ (อ้วน)
 - จงเขียนรหัสจำลองและวาดผังงาน ISO 5807 ของระบบนี้
- จงสร้างตาราง Trace Table สำหรับติดตามการทำงานของอัลกอริทึมต่อไปนี้ เมื่อป้อนค่า $N = 4$:

```
START
  INPUT N
  SET factorial = 1
  FOR i = 1 TO N DO
    SET factorial = factorial * i
  ENDFOR
  OUTPUT factorial
END
```

ตอนที่ 3 การสังเคราะห์และคิดขั้นสูง

- จงเขียนโปรแกรมภาษา Python 3.11 รับค่าตัวเลขจำนวนเต็ม N แล้วแสดงผลลัพธ์ว่า N เป็น "จำนวนเฉพาะ (Prime Number)" หรือไม่ โดยใช้โครงสร้าง While Loop พร้อมสร้างตาราง Trace Table แสดงการทดสอบตัวหารทุกตัวตั้งแต่ 2 ถึง $\sqrt{N}$

เอกสารอ้างอิงประกอบ

1. Böhm, C., & Jacopini, G. (1966). Flow diagrams, turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5), 366–371.
2. International Organization for Standardization. (1985). *Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts* (ISO Standard No. 5807:1985).
3. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
4. NASA. (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*. National Aeronautics and Space Administration.
5. Thassana, C. (2026). *Computational Thinking and Applied Artificial Intelligence for Science Education*. Rambhai Barni Rajabhat University Press.